

Programlamaya Giriş: Hafta 2

- Değişkenleri "=" (eşittir) işareti ile atıyoruz.
 - `int x = 10` ifadesi `x` isimli değişkene 10 değerini atar.
 - `char b= "ali"`
- int x = 4 * 7 + a * 2**
-
- Sol Taraf**
Değerin veri tipi
Değerin saklanacağı ad.
- Sağ Taraf**
Atanmak istenen/elde edilen değer.
- Atama Operatörü**
Değerin nasıl saklanacağını tanımlar

Değişkenlerle Sık Yapılan Hatalar;

- Başlatılmadan (initialize edilmeden) değişken kullanımı.
- Aynı değişkeni iki kez bildirmek.
- Uyuşmayan veri tipi kullanımı.

Atama Operatörleri;

En yüksek öncelik kuralından en düşük öncelik kuralına göre;

- Parantezlere her zaman uyulur
- Üs Alma (Üse Alma)
- Çarpma, Bölme ve Kalan
- Toplama ve Çıkarma
- Soldan sağa

Format Kontrol Dizgesi

İş

int yazdır

flout yazdır

flout oku

int oku

double oku

tek karakter oku

Doğru kullanım

```
printf("%d", x);
```

```
printf("%.2f", x);
```

```
scanf("%f", &x);
```

```
scanf("%d", &x);
```

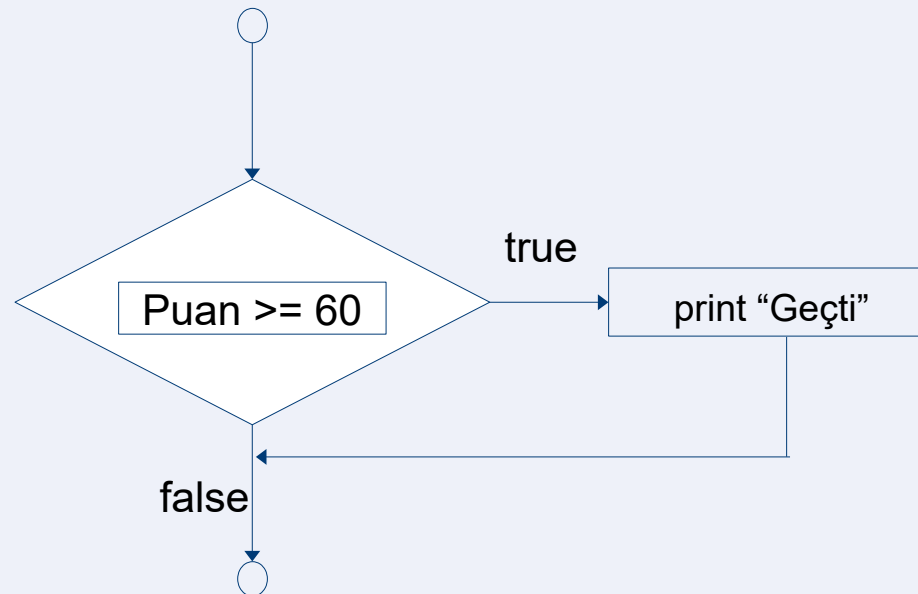
```
scanf("%lf", &x);
```

```
scanf("%c", &ch);
```

If-Akış Diyagramı ve Doğruluk

■ Sözde kod:

- Karar herhangi bir ifade üzerine kurulabilir.
- Koşul: $\text{puan} \geq 60 \rightarrow \text{true}$ ise "Geçti" yazdır, false ise atla.



if Biçimleri

- *if (condition)*
 statement;
- *if (condition)*
 statement;
 else
 statement;

if..else Seçim İfadesi

Eğer not ≥ 60 ise "Passed" yazdır

Değilse "Failed" yazdır

```
■ if (grade  $\geq$  60)  
    printf("Passed\n");  
else  
    printf("Failed\n");
```

else-if Merdiveni

- Örnek:

```
if (condition1)  
    statement;  
else if (condition2)  
    statement;  
else if (condition3)  
    statement;  
else  
    statement;
```


else-if Merdiveni

■ Örnek:

```
int not = 72;  
if (not >= 90) printf("A\n");  
else if (not >= 80) printf("B\n");  
else if (not >= 70) printf("C\n");  
else printf("D/F\n");
```

İç İçe if (Nested if)

- İç içe if..else: Birden fazla durumu test etmek için iç içe yerleştirilebilir.
- Koşul karşılandığında, kalan dallar atlanır.
- Uygulamada çok derin girintilemeden kaçınılır.

İç İçe if (Nested if)

- Örnek:

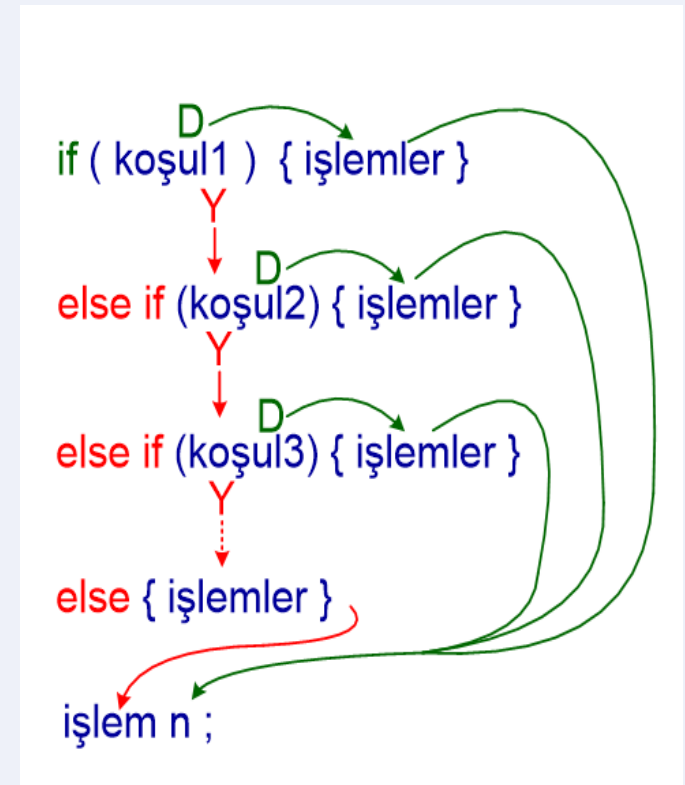
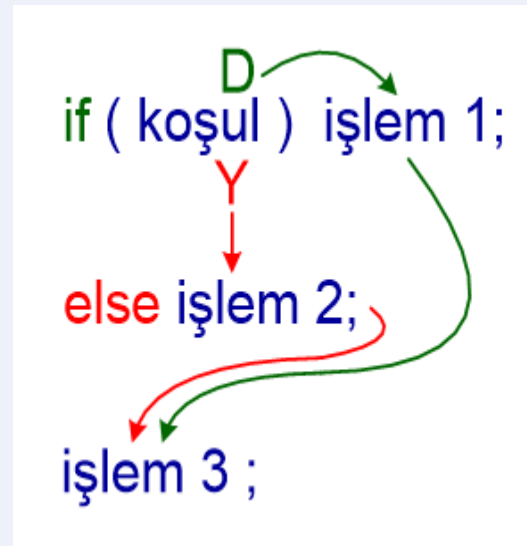
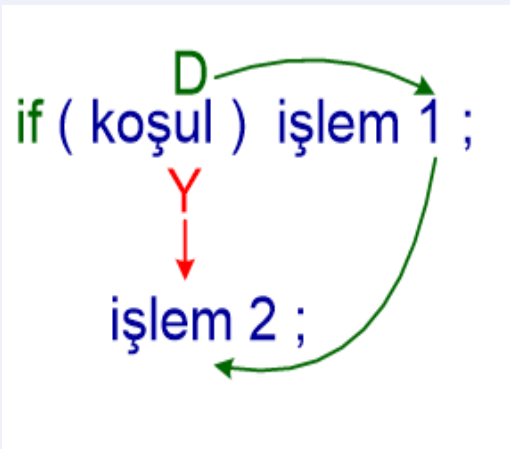
```
if (condition1)  
    if (condition2)  
        statement1;  
    else  
        statement2;  
else  
    statement3;
```

İç İçe if (Nested if)

```
int a = 10, b = 20;  
if (a > 0) {  
    if (b > 0)  
        printf("İkisi de pozitif\n");  
}
```

Algoritma ve Programlama

if-else Yapıları: Mantıksal bir ifadenin doğru yada yanlış olmasına göre farklı yerlere dallanmayı sağlar.



if..else Seçim İfadesi

- `a = 3` iken `x` ve `a` kaç olur?
- `int x = (a++ + 2 > ++a) ? a : (a - 1);`
- `a++` (Post-increment): Mevcut değeri (3) işleme sokar, ancak işlem bittikten sonra `a`'yı 1 artırır. Yani burada kullanılan değer 3'tür.
- `++a` (Pre-increment): `a` değerini hemen 1 artırır ve yeni değeri işleme sokar.

if..else Seçim İfadesi

- `int a = 3; int y = a++;`
- `int a = 3; int y = ++a;`

Eğer `not >= 60` ise "Passed" yazdır Değilse "Failed" yazdır
Üçlü koşullu operatör (?:)

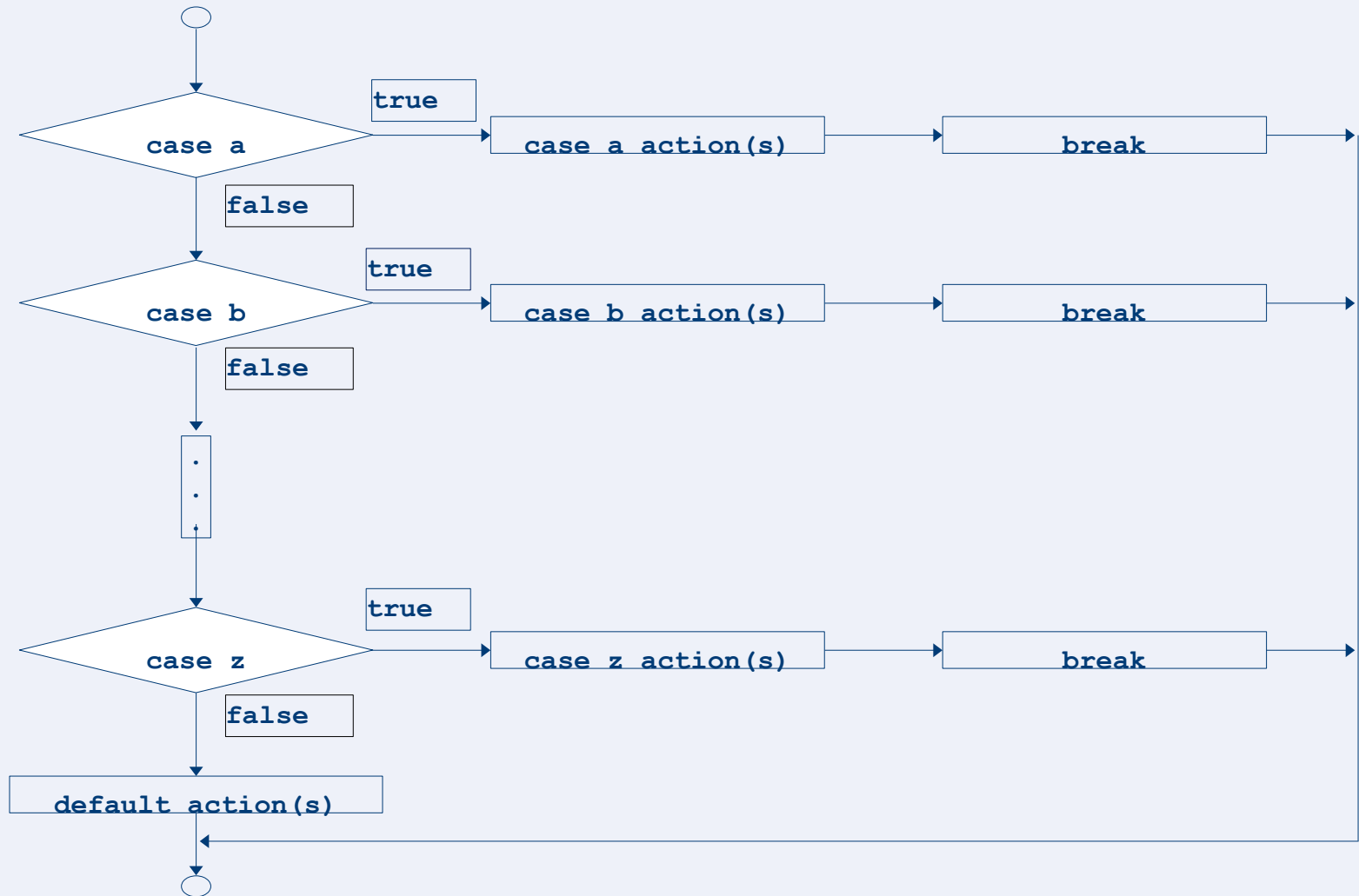
- Üç argüman alır (koşul, doğruysa değer, yanlışsa değer)
- `printf("%s\n", grade >= 60 ? "Passed" : "Failed");`
- `grade >= 60 ? printf("Passed\n") : printf("Failed\n");`

switch Deyimi

■ Amaç:

- Bir **if-else merdivenini** daha okunur hâle getirmek;
- Tek bir ifadenin **ayrık değerlerine** göre dallanmak.

switch Deyimi



switch Deyimi

- Genel Biçim:

```
switch (expr) { // expr: tamsayı türleri, char,enum  
  
    case sabit1: /* ... */ break;  
    case sabit2: /* ... */ break;  
    /* ... */  
    default:    /* ... */ break;  
}
```

```
char op;  
int a = 5, b = 3;  
  
switch (op) {  
    case '+': printf("%d\n", a+b); break;  
    case '-': printf("%d\n", a-b); break;  
    default:  printf("Bilinmeyen islem\n");  
break;  
}
```

switch Deyimi

- return 0 ve return 1 ne anlama gelir?
 - main fonksiyonunda return 0; → programın başarıyla bittiğini işletim sistemine bildirir.
 - main fonksiyonunda return 1; (ve genel olarak 0 dışı değerler) → hata/başarısızlık durumu demektir.

switch Deyimi

- return 0 ve return 1 ne anlama gelir?

```
#include <stdio.h>
```

```
int main(void) {  
    printf("A");  
    return 0;  
    printf("B");  
}
```

switch Deyimi

- return 0 ve return 1 ne anlama gelir?

```
#include <stdio.h>
```

```
int main(void) {
```

```
    FILE *f = fopen("data.txt", "r");
```

```
    if (!f) {
```

```
        printf("Dosya bulunamadi!\n");
```

```
        return 1; // Program çalıştı ama görev başarısız: dosya açılmadı
```

```
    }
```

```
    printf("Dosya acildi, islem basliyor...\n");
```

```
    fclose(f);
```

```
    return 0; // Her şey yolunda
```

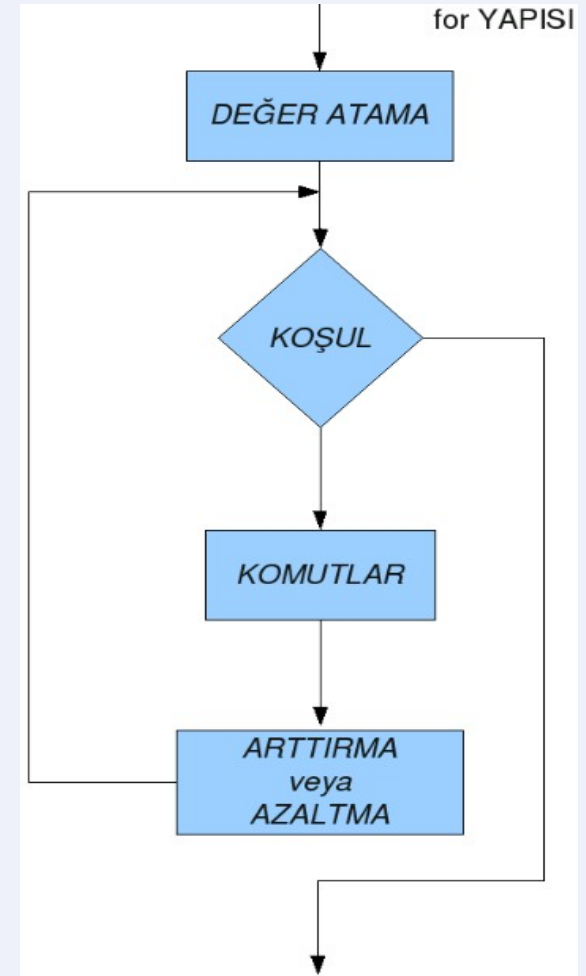
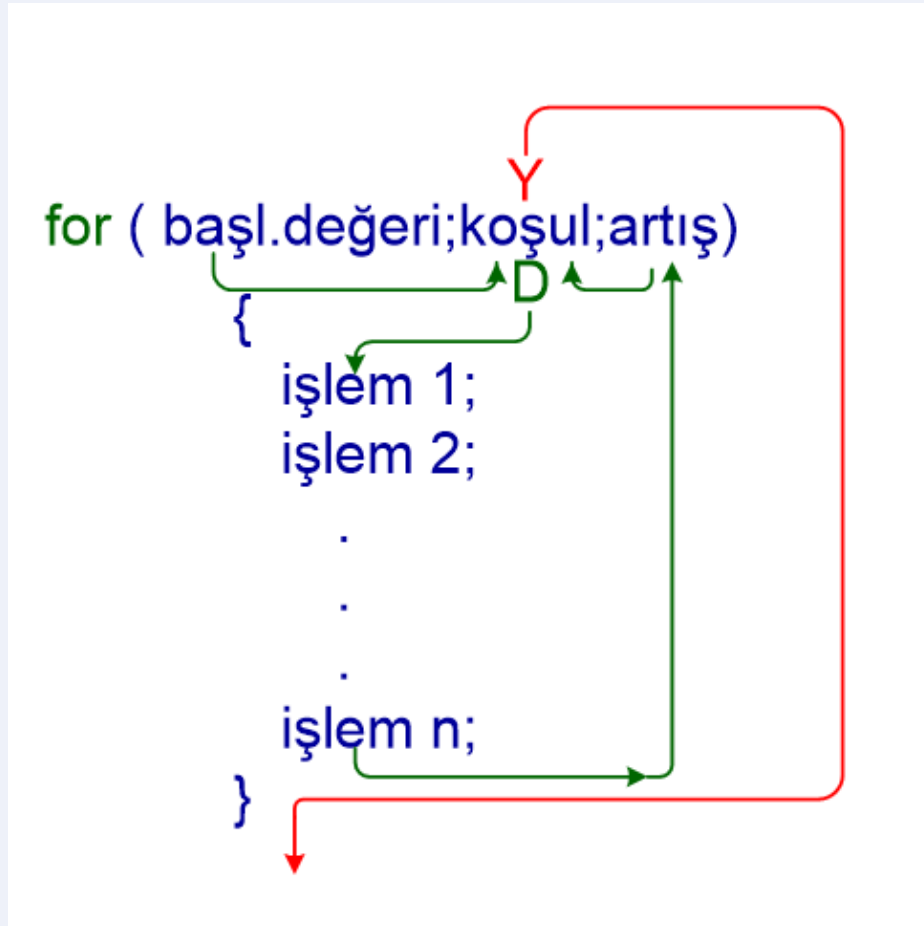
```
}
```

for Döngüsü:

- Tek bir işlem satırını veya birden fazla işlem satırından oluşan bloğun istenen sayıda tekrarlanması için kullanılır.
- Sayıcı döngü komutudur. Örneğin döngünün N defa çalışmasını istiyoruzdur.
- Daha sık kullanılır.
- Koşul sağlandığı (doğru olduğu) sürece döngü bloğu işlemleri yapılır.

Algoritma ve Programlama

for Döngüsü:



for Döngüsü:

Aritmetik ifadeler

Başlatma, döngü devamı ve artış, aritmetik ifadeler içerebilir. $X=2$ ve $y=10$ ise

$\text{for} (j = x; j \leq 4 * x * y; j += y / x)$ şuna eşdeğerdir
 $\text{for} (j = 2; j \leq 80; j += 5)$

"Artış" negatif olabilir (azaltma)

Döngü devam koşulu başlangıçta yanlışsa for ifadesinin gövdesi gerçekleştirilmez

Kontrol, for ifadesinden sonraki ifadeye geçer

İç içe Döngüler:

-Bir döngü (loop) bir dönüşünü tamamlamadan diğer bir veya birkaç döngü dönme işlemi gerçekleştirebilir.

```
for (dış = 0; dış < N; dış++) {  
    for (iç = 0; iç < M; iç++) {  
        // İç döngü işlemleri  
    }  
}
```

Dış döngü başlar

İç döngü tüm turu tamamlar

Dış döngü bir sonraki tura geçer

Bu işlem dış döngü bitene kadar sürer

Dış döngü 3 kez, iç döngü 5 kez

çalışıyorsa toplam 15 işlem olur

Algoritma ve Programlama

for Döngüsü:

Örnek

```
#include <stdio.h>
```

```
int main() {  
    int x, i, j;
```

```
    for (x = 1; x < 2; x++) {  
        for (j = 1; j < 3; j++) {  
            for (i = x; i < x + 3; i++)  
                printf("%d\n", i * j);  
        }  
    }
```

```
    return 0;
```

```
}
```

x	j	i	i*j	Ekran Çıktısı
1	1	1	1	1
1	1	2	2	2
1	1	3	3	3
1	2	1	2	2
1	2	2	4	4
1	2	3	6	6

Algoritma ve Programlama

İç içe Döngüler:

```
#include <stdio.h>
int main() {
    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            printf("%d\t", i * j);
        }
        printf("\n");
    }
    return 0;
}
```

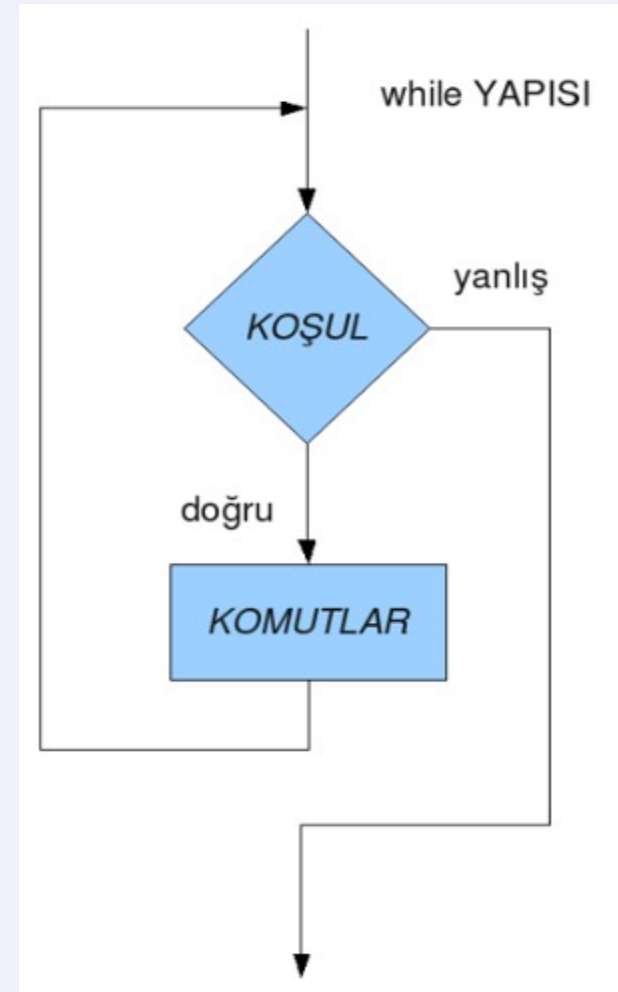
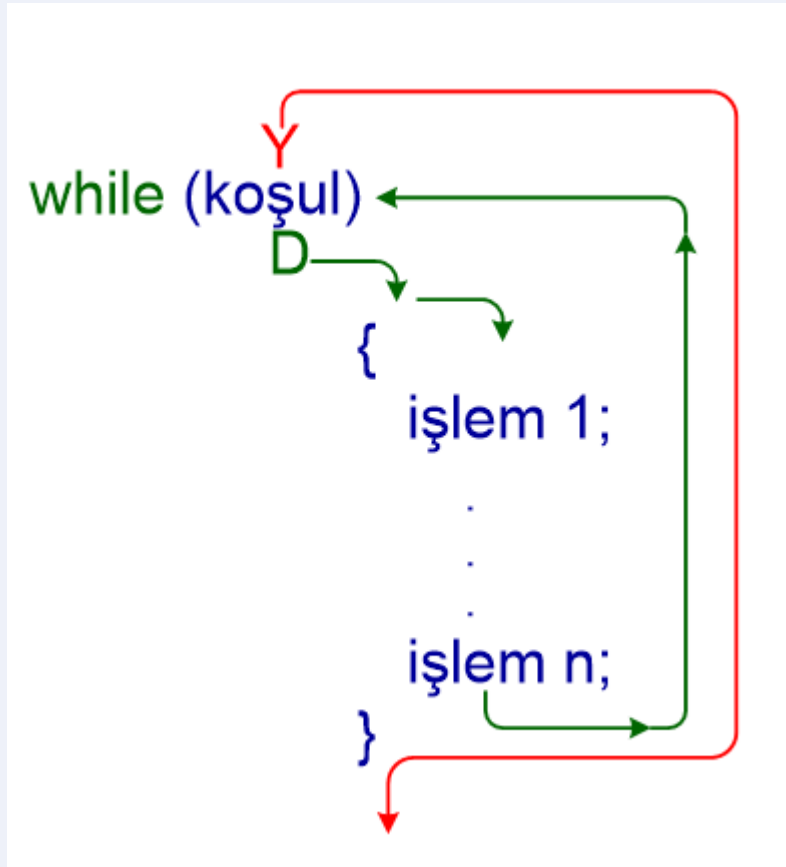
Adım	i	j	İşlem	Ekrana Yazılan
1	1	1	1×1	1
2	1	2	1×2	2
3	1	3	1×3	3
4	—	—	Satır bitti → \n	Yeni satır
5	2	1	2×1	2
6	2	2	2×2	4
7	2	3	2×3	6
8	—	—	Satır bitti → \n	Yeni satır
9	3	1	3×1	3
10	3	2	3×2	6
11	3	3	3×3	9

while Döngüsü:

- “mantıksal ifade” ile belirtilen koşul doğru olduğu sürece blok içerisindeki satırlar tekrarlanır. Koşul yanlış olduğu anda döngüden çıkar.
- C dilindeki ön koşullu döngüdür.
- Verilen koşul sağlandığı (doğru olduğu) sürece döngü içindeki işlemler gerçekleştirilir.
- Döngünün kaç defa çalışacağı bilinmediği durumlarda kullanılır.

Algoritma ve Programlama

while Döngüsü:



while Döngüsü:

Örnek

```
#include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    i = 1;
```

```
    while (i < 20) {
```

```
        printf("\ni = %d", i);
```

```
        i++; // i değerini 1 artır
```

```
    }
```

```
    printf("\nDongu bitti. i artik %d oldu (bir sonraki deger).", i);
```

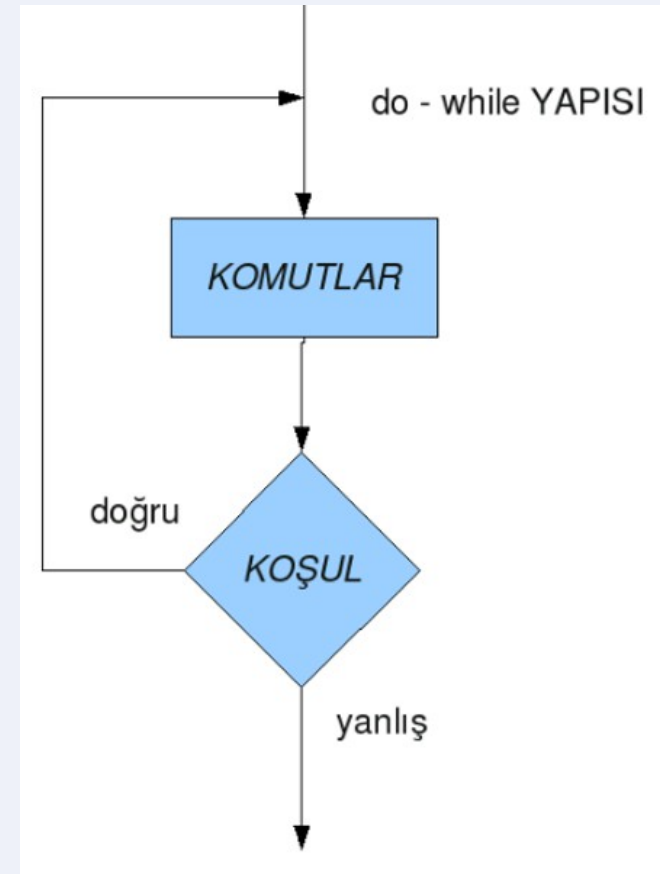
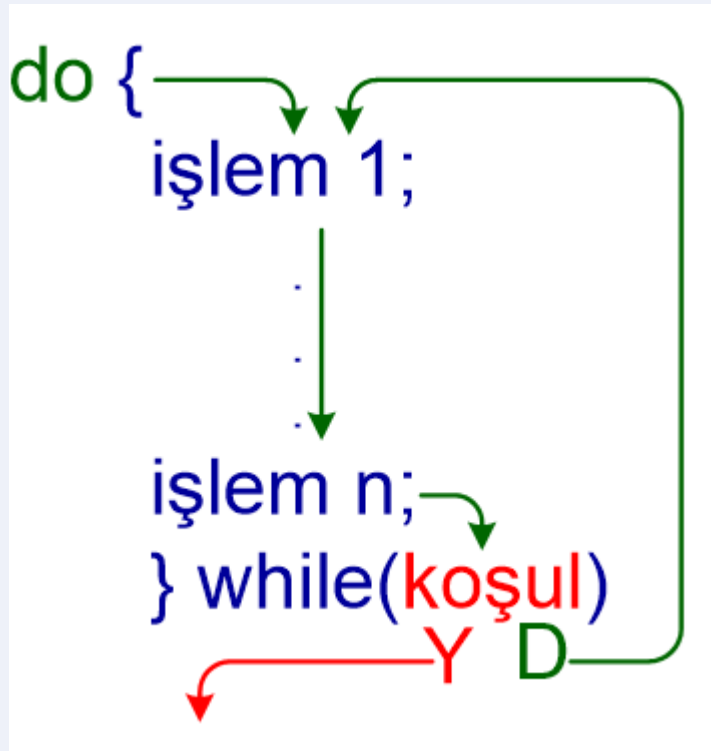
```
    return 0;
```

```
}
```

do - while Döngüsü:

- While döngüsü gibidir. Ancak do-while döngüsünde önce program doğrudan döngüye girer ve satırları çalıştırır. Daha sonra koşula bakılır. Koşul doğru olduğu sürece döngü bloğu tekrarlanır.
- Son koşullu döngüdür.
- "while" ile belirtilen koşul sağlandığı (doğru olduğu) sürece döngüdeki işlemler yapılır.
- Bazı durumlarda döngü bir kere çalıştıktan sonra devam edip etmemeye karar vermek isteriz. Bu durumlarda "do while" döngüsü kullanılır.
- Döngünün gövdesi en az bir kere çalışmaktadır.

do - while Döngüsü:



do – while Döngüsü:

Örnek

```
#include <stdio.h>
```

```
int main() {  
    int i;
```

```
    i = 1;
```

```
    do {
```

```
        printf("\ni = %d", i);
```

```
        i++;
```

```
    } while (i < 20);
```

```
    printf("\nDongu bitti. i'nin bir sonraki degeri = %d", i);
```

```
    return 0;
```

```
}
```

Örnekler

İf koşul:

-Kullanıcıdan bir öğrencinin vize ve final notunu al. Ağırlıklı ortalama ile başarı durumunu yazdır:

- Ortalama = $0.4vize + 0.6final$
- Ortalama ≥ 60 ise “Geçti”, değilse “Kaldı”
- Ayrıca final < 50 ise ortalama kaç olursa olsun “Final barajı: kaldı” yaz.

İf koşul:

■ işlem adımları

- vize, final oku
- ortalamayı hesapla
- $\text{final} < 50$ mi kontrol et (baraj)
- değilse $\text{ortalama} \geq 60$ mı kontrol et
- sonucu yazdır

■ Pseudocode

- read vize, final
- $\text{avg} = 0.4\text{vize} + 0.6\text{final}$
- if $\text{final} < 50 \rightarrow \text{print "Baraj"}$
- else if $\text{avg} \geq 60 \rightarrow \text{print "Geçti"}$
- else $\rightarrow \text{print "Kaldı"}$

İf koşulu:

```
#include <stdio.h>
```

```
int main(void) {  
    int vize, final;  
    double ort;  
  
    printf("Vize notu: ");  
    scanf("%d", &vize);  
  
    printf("Final notu: ");  
    scanf("%d", &final);  
  
    ort = 0.4 * vize + 0.6 * final;  
  
    // Önce baraj kontrolü (kural önceliği)  
    if (final < 50) {  
        printf("Final barajı nedeniyle KALDI. Ortalama = %.2f\n", ort);  
    } else if (ort >= 60) {  
        printf("GECTI. Ortalama = %.2f\n", ort);  
    } else {  
        printf("KALDI. Ortalama = %.2f\n", ort);  
    }  
  
    return 0;  
}
```

Switch case:

- Basit hesap makinesi: kullanıcı a, b ve işlem karakterini (+ - * /) girsin.
- switch ile sonucu yazdır.
- bölmede $b \neq 0$ ise hata ver.

Switch case:

■ işlem adımları

- a,b,op oku
- switch(op): +,-,*,/
- / için b==0 kontrol et

■ Pseudocode

- read a,b,op
- switch op: compute;
- if division and b==0 error

Switch case:

```
#include <stdio.h>
```

```
int main(void) {  
    double a, b;  
    char op;  
  
    printf("a b op(+ - * /): ");  
    scanf("%lf %lf %c", &a, &b, &op);  
  
    switch (op) {  
        case '+': printf("Sonuc = %.2f\n", a + b); break;  
        case '-': printf("Sonuc = %.2f\n", a - b); break;  
        case '*': printf("Sonuc = %.2f\n", a * b); break;  
        case '/':  
            if (b == 0) printf("Hata: sifira bolme!\n");  
            else printf("Sonuc = %.2f\n", a / b);  
            break;  
        default:  
            printf("Gecersiz islem!\n");  
    }  
  
    return 0;  
}
```

for:

- 1'den N'e kadar sayıların toplamını
- ve tek sayıların toplamını hesapla.

for:

■ işlem adımları

- N oku
- for i=1..N
- toplam += i
- i tekse tek
- Toplam += i

■ Pseudocode

- read N
- sum=0, odd=0
- for i=1..N: sum+=i;
- if i%2==1 odd+=i

for:

```
#include <stdio.h>
```

```
int main(void) {  
    int i,N;  
    int sum = 0, oddSum = 0;  
  
    printf("N: ");  
    scanf("%d", &N);  
  
    for (i = 1; i <= N; i++) {  
        sum += i;  
        if (i % 2 == 1) oddSum += i;  
    }  
  
    printf("Toplam = %d\n", sum);  
    printf("Teklerin toplami = %d\n", oddSum);  
  
    return 0;  
}
```

while:

- Kullanıcı 0 girene kadar sayı alsın.
- Girilen sayıların toplamını ve
- kaç tane sayı girildiğini yazdır.

while:

■ işlem adımları

- toplam=0, sayac=0
- sayı oku
- sayı 0 değilse
- döngü: ekle, sayacı artır, tekrar oku

■ Pseudocode

- sum=0,count=0
- read x
- while x != 0: sum+=x; count++; read x
- print sum,count

while:

```
#include <stdio.h>
```

```
int main(void) {  
    int x;  
    int sum = 0, count = 0;  
  
    printf("Sayi gir (bitirmek icin 0): ");  
    scanf("%d", &x);  
  
    while (x != 0) {  
        sum += x;  
        count++;  
  
        printf("Sayi gir (bitirmek icin 0): ");  
        scanf("%d", &x);  
    }  
  
    printf("Adet = %d, Toplam = %d\n", count, sum);  
    return 0;  
}
```

do while:

- Kullanıcıdan 1–100 arasında bir sayı iste.
- Geçerli girene kadar tekrar sor.
- (en az 1 kez sormak için do-while)

do while:

■ işlem adımları

- do: sayı al
- sayı aralıkta değilse uyar
- while sayı geçersiz

■ Pseudocode

- do:
 - read x
 - if invalid print warning
- while invalid
- print accepted

do while:

```
#include <stdio.h>
```

```
int main(void) {  
    int x;  
  
    do {  
        printf("1-100 arasi sayi gir: ");  
        scanf("%d", &x);  
  
        if (x < 1 || x > 100) {  
            printf("Gecersiz! Tekrar deneyin.\n");  
        }  
    } while (x < 1 || x > 100);  
  
    printf("Kabul edildi: %d\n", x);  
    return 0;  
}
```

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int i, s = 0;
```

```
    for (i = 1; i <= 6; i++) {
```

```
        if (i % 2 == 0) continue;
```

```
        s += i;
```

```
        if (s > 6) break;
```

```
        printf("%d ", s);
```

```
    }
```

```
    printf("| i=%d s=%d\n", i, s);
```

```
    return 0;
```

```
}
```

```
int x = 3, y = 0;

do {
    switch (x) {
        case 3: y += 2;          // break yok -> fall-through
        case 2: y += 3; break;
        default: y += 10;
    }

    x--;
    printf("%d ", y);
} while (x > 0);
```